APPLIED MACHINE LEARNING · CASE STUDY

Teaching a small model to speak your schema

What a real distillation-and-fine-tuning run teaches about constrained decoding, data hygiene, GPU scheduling, and why your measurements are the first thing you should distrust.

Solvr.ai · 22 August 2026 Qwen3-4B ← Kimi K3

1,103 examples · $22 · 0.182% of weights trained

01

The problem is a shape, not intelligence

An app needs JSON in an exact form. The model needs to produce it every time, not usually.

Solvr turns a maths problem into four things: worked steps, a lesson, formulas, and video suggestions. Each is a typed structure the app decodes. A model that solves the maths correctly but returns it in the wrong shape is, from the app's point of view, broken.

That reframes the goal. We are not chasing a smarter model. We want a **small** model that is reliable about **our** output format — and the two are almost unrelated problems.

WHAT HAPPENED HERE

Apple's on-device model, asked explicitly and repeatedly for LaTeX, returned `∫[0,1] 3x^2 dx` instead of `\int_{0}^{1} 3x^{2}\,dx`. The mathematics was correct. Firmer prompt wording changed the flavour of the wrong answer, not its correctness. Its weakness was presentation, and prompting was not a strong enough lever.

02

Two ways to get a shape

You can ask for a shape and repair what comes back, or you can make the wrong shape impossible to emit. These are not equivalent.

Repair after the fact

Ask for JSON in the prompt, then clean up: strip markdown fences, trim prose, fix escapes. This works, and it hides a nasty class of bug.

THE TRAP

`"\frac{d}{dx}"` is **valid JSON** — `\f` is a form feed. So a strict parser *succeeds* and hands back `<FF>rac{d}{dx}`. Every fraction in the app was silently corrupted, with no error raised anywhere. A response that parses is not the same as a response that parsed correctly.

Constrain the decoder

Constrained decoding restricts the tokens the model may emit so that output must satisfy a schema. The wrong shape is not merely discouraged; it is unreachable.

```
"response_format": {"type": "json_schema",
  "json_schema": {"name":"steps","strict":true,"schema":{…}}}
```

The important consequence is where failure goes. Repair can succeed and be wrong. Constrained generation either gives you a valid structure or throws. It does not remove failure — it removes *silent* failure.

BACKEND	DECODING	REPAIR LAYER
Hosted API, prose JSON	unconstrained	`load-bearing`
Hosted API + json_schema	constrained	`unreachable`
Apple on-device, guided	constrained	`impossible to need`
Self-hosted `mlx_lm.server`	unconstrained	`load-bearing`

GENERALISE

Schema compliance is a property of the **decoding mechanism**, not of model quality. The repair layer is not the price of using a hosted model — it is the price of asking *any* model for structure in prose. Note the last row: self-hosting is where you *lose* the guarantee.

03

Distillation: a big model teaches a small one

Knowledge distillation trains a small **student** on outputs produced by a large **teacher**. Here: Kimi K3 answered 1,200 prompts, and those answers became the training targets for a 4-billion-parameter Qwen3.

Two design decisions did most of the work.

- **The prompt in each training example is byte-identical to what the app sends.** Train on a paraphrase and you have fine-tuned for a prompt that never occurs in production.
- **Each target was produced by encoding the app's own decoded type.** So every target is, by construction, something the app can parse. You cannot accidentally teach the model to emit something you cannot read.

The student inherits the teacher — every weakness in step decomposition or notation is copied. It is the one decision that later engineering cannot fix, so it is worth measuring rather than assuming. Here: 3,788 of 3,789 LaTeX strings rendered, and five hand-checked solutions were mathematically correct.

> **ON HONESTY**
> Distillation from a commercial model is a legitimate, widely used method. It is also something people quietly do and loudly deny. State it: the write-up should name the teacher, the sample size, and the cost.

04

The dataset is where the bugs live

Three separate mistakes in building 1,103 examples. None were in the model code; all three would have degraded the result silently.

Garbage targets

A template emitted `Factor completely: x² + 7x + 23`. That does not factor over the integers. The teacher will produce *something*, and the student will learn to invent factorisations. Fixed by generating quadratics from their roots, so a clean answer is guaranteed by construction.

Hidden imbalance

Balancing the problem set by *topic* looked right and was not: 56 of 60 algebra problems came from a single template. Technically on-topic, useless for teaching variety. Balancing across *templates* took the set from 2 distinct shapes to 44.

Leakage across the split

Each problem yields four examples, one per surface. Splitting randomly over *examples* puts the steps for a problem in training and its lesson in test — so the held-out score

measures recall, not generalisation. The split must be by **problem**.

THE TRAP

The splitter *claimed* to group by problem: it hashed the prompt. But each surface has a different prompt, so every group held exactly one example and the split was random after all. The bug was visible only in a number nobody had thought to print — `examples per problem`. Once printed: `{2: 8, 3: 82, 4: 210}`, and 60 problems held out whole.

05

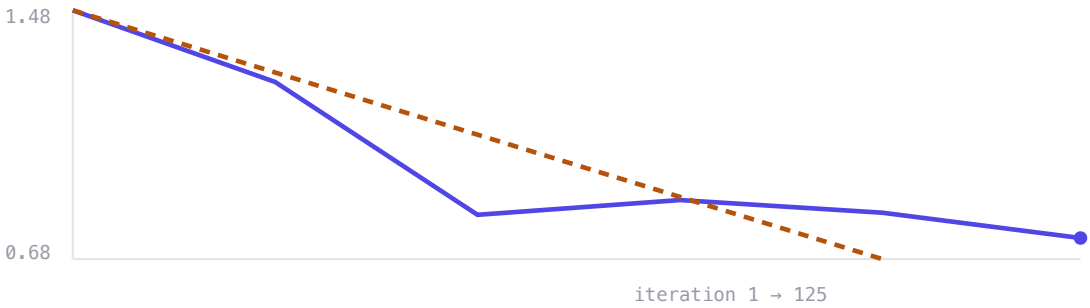
LoRA: train 0.182% of the weights

Full fine-tuning updates every parameter. **LoRA** — low-rank adaptation — freezes the model and learns a small additive correction to chosen weight matrices:

$$W' = W + BA, \qquad B \in \mathbb{R}^{d\times r},\ A \in \mathbb{R}^{r\times k},\ r \ll \min(d,k)$$

Because r is tiny, BA has far fewer parameters than W while still shifting behaviour. Applied to the last 16 of 36 blocks, that was **7.34M trainable parameters out of 4.02B — 0.182%**. The adapter is a few megabytes; the base model is untouched and shared.

This suits the goal exactly. We are not adding knowledge, we are adjusting output form, and form lives near the output.



Solid: training loss, 1.476 → 0.712. **Dashed:** validation loss, 1.476 → 0.685. Validation falling with training — rather than flattening or rising — is the signal that the model is learning the format instead of memorising the examples. 875 training examples, 114 validation, 113 test, split by problem.

06

The systems layer bites back

Training died three times with:

```
[METAL] Command buffer execution failed:
Impacting Interactivity
(kIOGPUCommandBufferCallbackErrorImpactingInteractivity)
```

A GPU driver batches work into a **command buffer**. macOS ends a buffer that monopolises the GPU for too long, to keep the interface responsive. This is not an out-of-memory error, and treating it as one sends you the wrong way: peak usage was 12.6 GB of 64.

It was diagnosed by *where* it died, not by guessing:

ATTEMPT	DIED IN	READING
`batch 4`	first validation (223 s)	one enormous buffer
`batch 2`	a training step	validation fixed; steps still too big
`batch 1 + accum 4`	iteration 125, intermittently	rare, not systematic

Gradient accumulation is the fix that costs nothing. Instead of one backward pass over 4 examples, run 4 passes of 1 and sum the gradients before stepping the optimiser. The gradient is $\nabla L = \frac{1}{4}\sum_{i=1}^{4} \nabla L_i$ either way — mathematically identical, but each command buffer holds a quarter as much work.

> **GENERALISE**
> When a failure is intermittent and environmental, stop hunting for the configuration that never triggers it and make triggering it cheap. Checkpoint every 25 iterations, resume from the last one, count progress cumulatively. A crash then costs two minutes instead of a run.

07

Distrust your instruments first

The most expensive errors in this project were in the things doing the measuring, not the things being measured.

A metric that flattered the wrong answer

Schema compliance was to be measured by a field recording whether a response needed repair. It compared the string *after* fence-stripping against the repaired one — so a response wrapped in a markdown fence recorded as clean. Prose trimming happened at the call site, outside the measurement entirely.

Both blind spots hid exactly what a weak model does most. A 4B that fenced every single response would have scored identically to a frontier model emitting bare JSON,

and the project's headline comparison would have been wrong in the direction that looks good.

An estimate built on the easy end

Cost was projected from a six-example probe. But the sampler took the *first* six problems, and the problem set was ordered by topic — so all six were simple algebra. Integrals and series reason far more, and reasoning tokens are billed as output.

	ESTIMATED	ACTUAL
Output tokens	909,000	1,505,133
Cost	$15.06	$22–24

GENERALISE

Sampling the head of an ordered list is not sampling. And an estimate that is wrong in a comfortable direction is worse than no estimate, because it gets believed. Prefer the ledger; label the guess as a guess.

08

What is actually known

At the time of writing the run is mid-flight, so here is the honest ledger.

CLAIM	STATUS
Teacher produces renderable, correct targets	Measured — 3,788/3,789 LaTeX render; 5/5 hand-checked correct
The student is learning	Measured — validation loss 1.476 → 0.685
The student beats anything on format	**Not yet measured.** Needs the eval harness
The student loses on reasoning	**Expected, unmeasured.** A 4B should
Cost per answer, local vs hosted	Unmeasured — energy needs the Mac on battery

A fine-tuned 4B that beats a frontier API on schema compliance and loses on reasoning is a real, publishable result — and considerably more credible than a claimed win. The point of building the harness before believing the model is that the harness is what makes the claim worth anything.